

Sistemas de Big Data

*Conforme a contenidos del «Curso de Especialización
en Inteligencia Artificial y Big Data»*



**Sistemas de
Big_Data**

Universidad de Castilla-La Mancha

Escuela Superior de Informática
Ciudad Real

8

Capítulo

Gestión y Almacenamiento de Datos - II

Julio Alberto López Gómez

La adopción de Hadoop como framework para el almacenamiento y procesamiento distribuido de grandes volúmenes de datos en entornos de Big Data ha producido la necesidad de contar con herramientas que permitan inyectar grandes volúmenes de datos en sistemas de ficheros distribuidos. Como respuesta a esta necesidad, el capítulo anterior describía la herramienta **Apache Flume** como servicio distribuido para recolectar, agregar y mover grandes cantidades de datos.

Si bien es cierto que **Apache Flume** estaba diseñado para trabajar con **datos no estructurados**, no es menos cierto que hay una necesidad de importación o ingestión de **datos estructurados** en sistemas de ficheros distribuidos. Estos datos se corresponden con aquellos provenientes, principalmente, de **bases de datos y almacenes de datos**. Con todos estos datos almacenados de forma local se ha trabajado tradicionalmente. Sin embargo, ahora es necesaria su ingestión en sistemas de ficheros distribuidos para obtener el máximo potencial de ellos.

Este capítulo describe y profundiza en la utilización de la herramienta **Apache Sqoop**, propia del ecosistema Hadoop, como plataforma distribuida y open-source para la **importación y exportación de datos estructurados** en sistemas de ficheros distribuidos.

8.1. Apache Sqoop: Introducción

Apache Sqoop¹ es una herramienta perteneciente al ecosistema Hadoop diseñada para la **importación y exportación de datos estructurados** en **sistemas de ficheros** distribuidos. Así, Sqoop es una herramienta que permite la ingesta de datos estructurados para su procesamiento en un framework de Big data, como por

¹<https://sqoop.apache.org>

ejemplo Hadoop. Por tanto, es posible utilizar Sqoop para importar datos de una base de datos relacional (MySQL u Oracle) o de un almacén de datos a HDFS, transformando así los datos para ser procesados por otros servicios como MapReduce. Una vez procesados, Sqoop también permite exportar los datos de vuelta a la base de datos o almacén de datos de origen.



Apache Sqoop: Plataforma distribuida, perteneciente al ecosistema Hadoop, open-source, escrita en Java y diseñada para la importación y exportación de datos estructurados en entornos de Big Data.

Apache Sqoop permite **automatizar** los procesos de **importación y exportación de datos**, teniendo en cuenta el esquema de los datos a importar según el diseño de la base de datos. Así pues, Sqoop permite la importación *bulk* o “a granel” lo cual posibilita la importación de bases de datos completas o de tablas individuales. De esta forma, Sqoop es capaz de **mapear e importar** bases de datos y datos contenidos en almacenes de datos en HDFS, HBase, Hive... así como exportarlos. Merece la pena destacar que Sqoop **utiliza MapReduce** para realizar las inyecciones de datos, por lo que las transferencias de datos se realizan **en paralelo, optimizando el rendimiento y consumo de recursos**, además de disponer de mecanismos de tolerancia a fallos. Así pues, Apache Sqoop garantiza un **análisis eficiente** de los datos, **reduciendo la sobrecarga** que produciría la **importación o exportación** de los datos realizada de forma **tradicional**. Para ello, Sqoop ofrece un **intérprete de comandos** para la interacción con el desarrollador, soportando la interacción de forma programada, **generando clases en Java**. De forma resumida, la figura 8.1 muestra de forma esquemática el funcionamiento de Apache Sqoop.

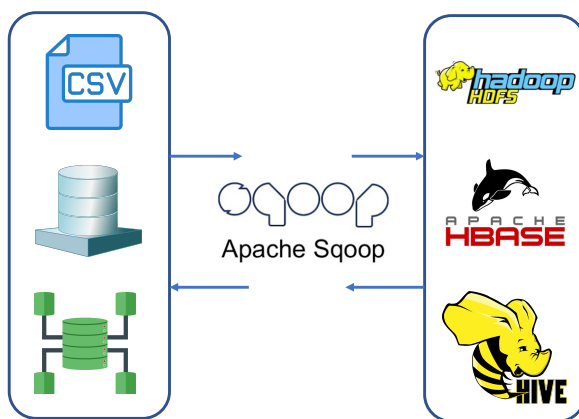


Figura 8.1: Esquema de funcionamiento de Apache Sqoop.

A pesar de que tanto **Apache Sqoop** como **Apache Flume** son dos herramientas de **ingestión de datos en sistemas de ficheros distribuidos**, existen **diferencias** significativas entre ambas que hacen que, en función de la aplicación, una u otra sea más adecuada. Estas diferencias están relacionadas, principalmente, con el **tipo de datos utilizado, el flujo de datos, la carga de los mismos y el caso de aplicación concreto**.

- **Tipo de datos:** Flume está concebido principalmente para trabajar con datos no estructurados y provenientes de varias fuentes, como aquellos que provienen de redes sociales, servidores web, logs... Por su parte, Sqoop está concebido para la ingestión de datos estructurados, provenientes de BBDD (MySQL, Oracle, Postgres) o almacenes de datos.
- **Flujo de datos:** Mientras que el flujo de datos que permite inyectar Apache Flume es un flujo de datos generados de forma continuada o streaming, el flujo de datos con el que trabaja Apache Sqoop es un flujo de datos permanente, como lo es el proveniente de una base de datos o almacén de datos que de ha de poseer conectividad JDBC.
- **Carga de los datos:** Flume posee una arquitectura dirigida por eventos, por lo que la carga de los datos se produce encapsulando los mismos en eventos y transmitiéndolos a través de una serie de canales hasta que se produce la ingesta. Por su parte, Sqoop no posee una arquitectura dirigida por eventos, sino que realiza una importación directa de los datos en el repositorio de destino.
- **Caso de aplicación:** Apache Flume está especialmente diseñado para ser utilizado en aplicaciones donde se pretenda recolectar y agregar datos, siendo necesario garantizar una alta disponibilidad de los mismos, así como fiabilidad y seguridad. En el caso de Apache Sqoop, esta herramienta está especialmente diseñada para trabajar en aplicaciones que realicen grandes importaciones y exportaciones de datos que requieran de la paralelización de estas operaciones.

8.2. Primeros pasos con Apache Sqoop

Apache Sqoop viene instalado y configurado por defecto en los servicios que ofrece la máquina virtual de **Cloudera** (<https://www.cloudera.com>). **Cloudera** es una empresa especializada en Big Data que ofrece soluciones de **gestión, almacenamiento y procesamiento** de datos a empresas y organizaciones. Entre sus productos, destaca **CDH** (Cloudera Distribution Hadoop). Se trata de una distribución Hadoop ampliamente utilizada a nivel empresarial y que incorpora un amplio conjunto de herramientas y servicios del ecosistema Hadoop como HBase, Mahout, Pig, Hive... La utilización de esta máquina virtual proporciona un entorno uniforme y controlado para la realización de pruebas y trabajo con Apache Hadoop y sus servicios.

En primer lugar, para comenzar a utilizar Sqoop, es necesario **arrancar los servicios de Hadoop**. Para ello, se puede ejecutar desde el escritorio de la máquina virtual “*Launch Cloudera Express*” o ejecutar el siguiente comando en un terminal.

```
sudo /home/cloudera/cloudera-manager - -force - -express
```

Una vez arrancados todos los servicios de Hadoop, es posible **comprobar que la herramienta Sqoop está instalada** utilizando el comando:

```
sqoop help
```

Este comando mostrará la ayuda del comando sqoop, que incluye todos los servicios disponibles de la herramienta. Por otra parte, es posible **mostrar la ayuda de sqoop sobre alguna de las herramientas**. Por ejemplo, si se quiere mostrar la ayuda sobre la utilidad *import* de sqoop, se puede utilizar el comando:

```
sqoop import --help
```

Una vez comprobado que Sqoop está correctamente instalado, el siguiente paso será **crear un usuario en MySQL** que será utilizado posteriormente para trabajar con Sqoop. Para ello, se escribe el siguiente comando:

```
sudo mysql -p
```

La ejecución de este comando pedirá la introducción de la contraseña para acceder en modo super-usuario. Una vez introducida y abierto el cliente MySQL, el siguiente comando permitirá crear un usuario para utilizar MySQL sin permisos de administrador:

```
CREATE USER 'username'@'localhost' IDENTIFIED BY 'password';
```

Donde *username* es el nombre del usuario que se ha creado y *password* es la contraseña de acceso a dicho usuario. Después de la creación del usuario, el siguiente paso es **asignar permisos** a dicho usuario para la edición de todos los archivos, lo cual es posible a través de la ejecución del comando:

```
GRANT ALL PRIVILEGES ON *.* TO 'username'@'localhost'
```

Donde, de nuevo, *username* es el nombre de usuario que se creó. Una vez creado el usuario, será posible acceder al cliente MySQL con el usuario creado, para ello, se cerrará la terminal en la que se han ejecutado los comandos anteriores y se abrirá una nueva, en la que se introducirá el comando:

```
mysql -u 'username' -p
```

Tras ejecutar el comando, se introducirá la contraseña del usuario *username* creado con anterioridad, accediendo así al cliente MySQL desde el usuario creado. A continuación, a modo de ejemplo, se muestran los comandos necesarios para **crear la base de datos *testDatabase*** y acceso a la misma:

```
CREATE DATABASE testDatabase;  
use testDatabase;
```

Una vez en la base de datos *testDatabase*, se **creará la tabla ciudades** por medio del código mostrado en el listado 8.1.

Listado 8.1: Creación de la tabla ciudades

```
1 CREATE TABLE 'ciudades' (  
2 'id' INTEGER UNSIGNED NOT NULL,  
3 'ciudad' VARCHAR(50),  
4 'region' VARCHAR(150),  
5 PRIMARY KEY ('id')  
6 );
```

Una vez creada la tabla, el listado muestra cómo **introducir algunos registros** dentro de dicha tabla.

Listado 8.2: Inserción de registros en la tabla ciudades

```
1 INSERT INTO 'ciudades' VALUES (1, "Ciudad Real", "Castilla la Mancha");  
2 INSERT INTO 'ciudades' VALUES (2, "Toledo", "Castilla la Mancha");  
3 INSERT INTO 'ciudades' VALUES (3, "Cuenca", "Castilla la Mancha");  
4 INSERT INTO 'ciudades' VALUES (4, "Albacete", "Castilla la Mancha");  
5 INSERT INTO 'ciudades' VALUES (5, "Gudalajara", "Castilla la Mancha");  
6 INSERT INTO 'ciudades' VALUES (6, "Caceres", "Extremadura");  
7 INSERT INTO 'ciudades' VALUES (7, "Badajoz", "Extremadura");
```

La realización de todos estos pasos permitirá tener una pequeña base de datos de prueba para, en secciones posteriores, comenzar a trabajar con Sqoop.

8.3. Importación de datos con Sqoop

La **importación de datos** es el proceso a través del cual Sqoop inyecta los datos provenientes de una base de datos o almacén de datos dentro de un sistema de ficheros distribuido (HDFS, HBase, Hive...). A la hora de realizar la importación, Sqoop permite importar **bases de datos completas** o también importar **solo algunas tablas** de una base de datos o almacén de datos.

El **patrón de diseño** que utiliza Apache Sqoop para realizar la importación de los datos en el sistema de ficheros distribuido correspondiente es el patrón **Change Data Capture (CDC)**. Este es un patrón de diseño que captura los cambios producidos en los datos, importando solo los cambios producidos en lugar de volcar todos los datos de nuevo. Por este motivo, la importación de datos realizada por Sqoop recibe el nombre de **importación incremental**.

Volviendo al ejemplo de la base de datos creada en el apartado anterior, el listado 8.3 muestra, en primer lugar, una plantilla del comando que permite importar la base de datos y justo a continuación, el código exacto que permite dicha importación, el cual puede ser ejecutado en la máquina virtual de cloudera.

Listado 8.3: Importación de la base de datos de prueba

```
1 sqoop import
2     -- connect <cadena_conexion>
3     -- username <nombre_usuario>
4     -- password <contraseña>
5     -- table <tabla_a_importar>
6
7 sqoop import
8     -- connect jdbc:mysql://localhost/testDatabase
9     -- username bda001
10    -- password 1234
11    -- table ciudades
```

Es posible comprobar si la importación se ha realizado correctamente mediante la utilidad *list_databases* de sqoop. El listado 8.4 muestra la plantilla de uso de esta utilidad así como el ejemplo concreto para el caso de estudio.

Listado 8.4: Uso de la herramienta *list_databases* de Sqoop

```
1 sqoop list_databases
2     -- connect <cadena_conexion>
3     -- username <nombre_usuario>
4     -- password <contraseña>
5
6 sqoop list_databases
7     -- connect jdbc:mysql://localhost
8     -- username bda001
9     -- password 1234
```

Análogamente, la herramienta *list_tables* cuya sintaxis es idéntica a *list_databases* permite obtener todas las tablas para una base de datos dada en la cadena de conexión. El listado 8.5 muestra la plantilla del uso de este comando de sqoop así como el ejemplo concreto para el caso de estudio.

Listado 8.5: Uso de la herramienta *list_tables* de Sqoop

```
1 sqoop list_tables
2     -- connect <cadena_conexion>
3     -- username <nombre_usuario>
4     -- password <contraseña>
5
6 sqoop list_tables
7     -- connect jdbc:mysql://localhost/testDatabase
8     -- username bda001
9     -- password 1234
```


Por otra parte, es posible importar los datos dentro de un directorio concreto en HDFS. Para ello, basta con utilizar la opción `--target-dir` incluyendo la ruta del directorio HDFS donde se van a inyectar los datos. El listado 8.6 muestra una plantilla genérica de importación al directorio `/user/test`. Además, en caso de que no existiera una clave primaria en la tabla, sería necesario utilizar la opción `--split-by <campo>` para indicar el campo que sqoop tomará como referencia de clave primaria dentro de la tabla.

Listado 8.6: Importación de datos a un directorio HDFS

```
1 sqoop import
2   -- connect <cadena_conexion>
3   -- username <nombre_usuario>
4   -- password <contraseña>
5   -- table <tabla_a_importar>
6   -- target-dir <ruta_directorio_HDFS>
7
8 sqoop import
9   -- connect jdbc:mysql://localhost/testDatabase
10  -- username bda001
11  -- password 1234
12  -- table ciudades
13  -- target-dir /user/test
```

Finalmente, también es posible importar los datos que son, a su vez, **resultado de la ejecución de una consulta**. Por ejemplo, es posible importar a un directorio concreto de HDFS aquellas ciudades de la región de Extremadura. El listado 8.7 muestra en primer lugar la sintaxis del comando y, en segundo lugar, el comando concreto para este caso práctico.

Listado 8.7: Importación de datos provenientes de una consulta a HDFS

```
1 sqoop import
2   -- query <SELECT FROM>where $CONDITIONS
3   -- where <condicion>
4   -- connect <cadena_conexion>
5   -- username <usuario>
6   -- password <contraseña>
7   -- split-by <clave_primaria>
8   -- target-dir <ruta_fichero>
9   -- fields-terminated by <separador>
10
11 sqoop import
12   -- query select id, ciudad, region
13   -- from ciudades where $CONDITIONS
14   -- where region=Extremadura
15   -- connect jdbc:mysql://localhost/testDatabase
16   -- username bda001
17   -- password 1234
18   -- split-by id
19   -- target-dir /user/test/
20   -- fields-terminated by \t
```

En el anterior listado se puede ver cómo aparecen tres opciones importantes: — *split-by* que indica qué campo será la clave primaria de los datos a importar, — *target-dir* que indica la ruta al directorio HDFS dentro del cual se importarán los datos y — *fields-terminatedby* que es la opción en la que se especifica cuál va a ser el separador de los datos importados. En este caso, se ha elegido el tabulador.

8.3.1. Importación incremental: Modo Append

A la hora de llevar a cabo la **importación de datos**, Apache Sqoop trabaja principalmente con dos formatos de archivos:

- **Texto delimitado:** Se trata del tipo de archivo por defecto que puede importarse utilizando Apache Sqoop. A la hora de importar datos desde este tipo de archivo, Sqoop escribe cada uno de los registros en forma de cadenas de caracteres, separando cada uno de los campos o columnas por medio de un carácter delimitador (espacio, coma, tabulador...)
- **Sequence files:** Formato de archivo binario que permite almacenar registros en tipos de datos personalizados. Cada uno de esos tipos de datos personalizados se corresponden con clases Java. La principal ventaja de este tipo de archivos es que los datos que contienen son soportados por otros módulos Hadoop, como MapReduce, lo que permite que estos datos sean utilizables por parte de otras herramientas del ecosistema Hadoop.

Por defecto, a la hora de importar los datos éstos **no se comprimen**, si bien es cierto que existen opciones que pueden utilizarse a la hora de ejecutar el comando *import* como son *-z*, *-compress* o *-compression-codec*.

Uno de los dos principales modos de importación de datos en Apache Sqoop es el **modo append**. Este es el modo de importación utilizado cuando se quieren importar datos para **añadir nuevas filas o registros a una tabla**. Cuando se utiliza este modo, **no se actualizan datos existentes**, sino que simplemente se añaden las nuevas filas o registros. Tomando como punto de partida el ejemplo de la tabla ciudades, supóngase que se añaden las siguientes ciudades, según lo especificado en el listado.

Listado 8.8: Nueva adición de registros en la tabla ciudades

```
1 INSERT INTO 'ciudades' VALUES (8, "Castellon", "Comunidad Valenciana");
2 INSERT INTO 'ciudades' VALUES (9, "Valencia", "Comunidad Valenciana");
3 INSERT INTO 'ciudades' VALUES (10, "Alicante", "Comunidad Valenciana");
```

A la hora de realizar la importación usando el modo append, es posible utilizar el comando que se muestra en el listado 8.9, donde aparece en primer lugar una plantilla para mostrar la estructura sintáctica del comando y, después, el comando concreto para este caso práctico.

Listado 8.9: Importación de datos modo append

```
1 sqoop import
2   -- connect <cadena_conexion>
3   -- username <nombre_usuario>
4   -- password <contraseña>
5   -- table <tabla_a_importar>
6   -- incremental append
7   -- check-column <campo>
8   -- last-value <valor>
9
10 sqoop import
11   -- connect jdbc:mysql://localhost/testDatabase
12   -- username bda001
13   -- password 1234
14   -- table ciudades
15   -- incremental append
16   -- check-column id
17   -- last-value 7
```

Como se puede observar, este comando incluye tres opciones adicionales a las de una importación usual, como es la que se incluye en el listado 8.3.

- **incremental:** Toma el valor “append” y define el modo de importación incremental que se va a utilizar.
- **check-column:** Identifica la columna o campo dentro de la tabla que se va a tomar como referencia para determinar si un registro se va a importar o no.
- **last-value:** Último valor de la columna o campo identificado por check-column y a partir del cual se importarán los siguientes registros.

Finalmente, para comprobar que efectivamente se han importado solo las nuevas ciudades insertadas, es posible ejecutar el comando **sqoop eval**, que permite evaluar una consulta sql. En este caso se tratará de una consulta para mostrar todos los registros de la tabla ciudades. El listado 8.10 muestra la sintaxis de sqoop eval, así como el comando utilizado en este caso práctico.

Listado 8.10: Utilización del comando sqoop eval

```
1 sqoop eval
2   --connect <cadena_conexion>
3   --username <nombre_usuario>
4   --password <contraseña>
5   --query <consulta>
6
7 sqoop eval
8   --connect jdbc:mysql://localhost/testDatabase --username bda001
9   --password 1234
10  --query "SELECT * FROM ciudades"
```

8.3.2. Importación incremental: Modo LastModified

El segundo modo de importación incremental es el modo **LastModified**. Este es el utilizado cuando se van a importar datos y dicha importación va a suponer, no solo añadir nuevas filas o registros, sino también **actualizar datos existentes**. La sintaxis es muy similar a la del modo `append`, con la única salvedad de que, en **LastModified**, la columna indicada por *check-column* suele ser de tipo **fecha y hora**, mientras que en el modo **append** solía ser de tipo **numérico**. Por último, en este modo, se importarán los datos añadidos y actualizados a partir del valor indicado en la opción *--last-value*. Sea el ejemplo de las ciudades y el caso en el que la ciudad de Badajoz se actualiza por Mérida según el comando:

```
UPDATE ciudades SET ciudad = "Merida" WHERE id = 7
```

A continuación, el listado 8.11 muestra cómo se produciría la importación utilizando el modo LastModified, con lo que únicamente se importaría el nombre de la ciudad de Mérida, que ha sido el único actualizado. Este listado muestra una plantilla con la sintaxis del comando de importación usando el modo incremental, así como el comando concreto para este caso práctico. Cabe destacar que, aunque en este caso la columna indicada por *check-column* no es de tipo fecha y hora, el comando funciona correctamente, lo cual puede ser comprobado por medio de *sqoop eval*. Sin embargo, es habitual, tal y como se mencionó anteriormente, que en el modo last modified, la columna indicada por *check-column* sea del tipo fecha y hora.

Listado 8.11: Importación de datos modo lastmodified

```
1 sqoop import
2   -- connect <cadena_conexion>
3   -- username <nombre_usuario>
4   -- password <contraseña>
5   -- table <tabla_a_importar>
6   -- incremental lastmodified
7   -- check-column <campo>
8   -- last-value <valor>
9
10 sqoop import
11   -- connect jdbc:mysql://localhost/testDatabase
12   -- username bda001
13   -- password 1234
14   -- table ciudades
15   -- incremental lastmodified
16   -- check-column id
17   -- last-value 1
```

8.3.3. Importación en HBase

En ocasiones es posible que HDFS no sea el repositorio o sistema de archivos de destino sino que, por ejemplo, el repositorio de destino sea **HBase**. En ese caso, es posible utilizar el comando **sqoop import**, solo que será necesario utilizar una serie de opciones propias de la importación en HBase.

Sea el ejemplo de la tabla ciudades, el listado 8.12 muestra en primer lugar la estructura sintáctica de la importación en HBase. En segundo lugar, el listado muestra el comando concreto para importar a una nueva tabla HBase aquellas ciudades que sean de Extremadura.

Listado 8.12: Importación de datos en HBase

```
1 sqoop import
2   -- connect <cadena_conexion>
3   -- username <nombre_usuario>
4   -- password <contraseña>
5   -- table <tabla_a_importar>
6   -- incremental lastmodified
7   -- check-column <campo>
8   -- last-value <valor>
9
10 sqoop import
11   -- connect jdbc:mysql://localhost/testDatabase
12   -- username bda001
13   -- password 1234
14   -- table ciudades
15   -- incremental lastmodified
16   -- check-column id
17   -- last-value 1
```

8.3.4. Importación en Hive

Hive es otro de los destinos en los que comúnmente se realizan importaciones de datos, utilizando Apache Sqoop u otra herramienta de ingesta de datos. Hive permite la lectura, gestión y acceso a almacenes de datos distribuidos.

Retomando el ejemplo de la tabla ciudades, el listado 8.13 muestra la estructura sintáctica de la importación en Hive mientras que, en segundo lugar, aparece el comando concreto para importar a una nueva base de datos aquellas ciudades que sean de Extremadura.

Listado 8.13: Importación de datos en Hive

```
1 sqoop import
2   -- query <consulta>
3   -- where <condicion>
4   -- connect <cadena_conexion>
5   -- hive-import
6   -- hive-database <nombre_db>
```

```
7      -- hive-table <nombre_tabla>
8
9  sqoop import
10     -- query SELECT id, ciudad, region FROM ciudades WHERE $CONDITIONS
11     -- where region = Extremadura
12     -- connect jdbc:mysql://localhost/testDatabase
13     -- hive-import
14     -- hive-database Extremadura
15     -- hive-table Ciudades
```

8.4. Exportación de datos con Sqoop

La **exportación** es el proceso inverso a la importación de datos, es decir, es el proceso mediante el cual Apache Sqoop permite **extraer ficheros de HDFS para almacenarlos de forma local en un SGBD**.

La exportación en Apache Sqoop se realiza de la siguiente forma: los ficheros de entrada son leídos y, después, son **parseados** a una serie de registros según los delimitadores especificados por el usuario. Merece la pena destacar que **la exportación solo será exitosa si la tabla destino de la exportación ya existe en la base de datos destino**. Por tanto, en caso de querer exportar una tabla que no exista en la base de datos a la cual se quiere exportar, se obtendrá un error en la exportación.

El proceso de parseo de los archivos de entrada consiste en **traducir estos archivos en una secuencia de sentencias INSERT que permita inyectar los registros en la base de datos de destino**. No obstante, también es posible realizar actualizaciones de los valores importados e incluso programar procedimientos para realizar la exportación de cada registro. Así pues, existen dos modos principales de exportación:

- **Update mode:** Se trata del modo que permite actualizar datos y/o reemplazar registros existentes cuando se realiza la exportación. Esto equivale al parseo de los archivos de entrada y la generación de las sentencias UPDATE correspondientes.
- **Call mode:** Este modo es el que permite la exportación de cada uno de los registros de forma personalizada, ya que cada uno de ellos se exporta atendiendo al procedimiento que fue programado para realizar la exportación. Por tanto, para cada registro se realiza una llamada a procedimiento que determina cómo se exportará dicho registro.

La herramienta de Sqoop que permite realizar la exportación de los datos es **sqoop-export**. Este comando de sqoop está concebido para exportar datos a una nueva tabla. En este punto, es muy importante tener en cuenta que, a la hora de exportar datos, el desarrollador **debe asegurarse de que los datos exportados cumplan con las restricciones de la tabla destino**. Así pues, es el desarrollador quien debe procurar que cuando se exportan datos a una base de datos ya existente, los datos exportados cumplan con las restricciones de integridad de la base de datos de destino.

Sea el siguiente ejemplo didáctico en el que, tal y como se puede apreciar en el listado 8.14 se ha creado una tabla llamada *foo* en una base de datos de prueba.

Listado 8.14: Creación de la tabla foo

```
1 CREATE TABLE foo(  
2     id INT NOT NULL PRIMARY KEY,  
3     msj VARCHAR(32),  
4     bar INT);
```

Por otra parte, se tiene un fichero HDFS tal y como el que se muestra en el listado 8.15.

Listado 8.15: Ejemplo de fichero HDFS

```
1 0,Texto de prueba,42  
2 1,Algún dato más,100  
3 ...
```

La sintaxis genérica del comando de exportación y la aplicación concreta de este comando al caso que se plantea se muestran en el listado 8.16.

Listado 8.16: Exportación

```
1 sqoop export  
2     --connect= <cadena_conexion>  
3     --username=<usuario>  
4     --password=<contraseña>  
5     --export-dir=<ruta_archivo_hdfs>  
6     --table=<nombre_tabla>  
7  
8 sqoop export  
9     --connect= jdbc:mysql://localhost/testDatabase  
10    --username= bda001  
11    --password= 1234  
12    --export-dir= /usr/test/archivo.csv  
13    --table= tabla_ejemplo
```

La ejecución del comando anterior implicaría el parseo del archivo de entrada y la traducción del mismo a sentencias SQL como las que se muestran en el listado 8.17.

Listado 8.17: Sentencias de exportación en lenguaje SQL

```
1 UPDATE foo SET msg='Esto es una prueba', bar=42 WHERE id=0;  
2 UPDATE foo SET msg='Algún dato más', bar=100 WHERE id=1;  
3 ...
```

Del mismo modo que las importaciones en Sqoop se realizan en paralelo, las exportaciones también. Esto es posible gracias a que una exportación es ejecutada por **múltiples procesos de escritura**, cada uno de los cuales utiliza su propia conexión a la base de datos. Además, la herramienta de exportación utiliza **multi-row INSERT** y, por lo tanto, por cada sentencia INSERT se introducen cien registros en la base de datos. No obstante, y a pesar de lo que pudiera parecer, la exportación no es un proceso atómico.

Aunque la exportación **no es un proceso demasiado complejo**, puede arrojar diferentes **errores**. Algunos de los más comunes son: pérdida de la conectividad desde el cluster Hadoop a la base de datos, la inserción de una fila que viola una restricción de integridad, como puede ser la regla de integridad de entidades, integridad referencial o cualquier otra restricción sobre un campo, el parseo de un registro del hDFS que es incorrecto o no está completo, el parseo de un registro utilizando delimitadores incorrectos y problemas de capacidad en lo que a memoria se refiere.

8.5. Gestión de Jobs con Sqoop

Cualquier aplicación Big Data requiere de la realización frecuente de **importaciones y exportaciones de datos**, ya que estas son dos operaciones básicas y fundamentales en entornos Big Data.

Un **job** en Apache Sqoop es una herramienta que permite **guardar la información de configuración requerida para ejecutar un comando sqoop tiempo más tarde**. Así pues, un job puede considerarse como una macro, es decir, una herramienta que va a permitir guardar la configuración de un comando que se ejecuta múltiples veces para ser ejecutado posteriormente de forma más fácil y sencilla para el desarrollador. El uso de jobs permite, por tanto, automatizar y facilitar la importación incremental de datos y la exportación de los mismos.

Cada vez que se crea un job en Apache Sqoop, las descripciones y configuraciones de los mismos se guardan en el directorio de sqoop. Concretamente, la configuración de los jobs se guarda en un espacio de almacenamiento llamado **metastore**. De esta forma, es posible compartir las configuraciones de los distintos jobs creados entre los usuarios de un mismo cluster, haciendo extensivos los servicios creados por algunos usuarios al resto y manteniendo la **disponibilidad** de estos servicios.

La creación de un job en apache Sqoop se realiza mediante la herramienta Sqoop job, la cual tiene una estructura sintáctica genérica como la que se aprecia a continuación

```
sqoop job (generic-args) (job-args) [--[subtool-name] (subtool-args)]
```

Donde (generic-args) son los argumentos genéricos de Hadoop y (job-args) son los argumentos propios del comando job. De forma opcional, se pueden indicar el nombre y los argumentos de las sub-herramientas que el job se va a encargar de ejecutar.

El argumento más importante de job es `--create`, el cual permite definir y almacenar un job específico. Por ejemplo, el siguiente comando permite definir un job para importar una tabla llamada `alumnos`.

```
sqoop job --create myjob --import --connect jdbc:mysql://localhost/testDatabase --table alumnos
```

Por tanto, el job *myjob* que acaba de ser creado se encargará de importar la tabla `alumnos` en la base de datos denominada `testDatabase`.

Para ejecutar este job tan solo hace falta ejecutar el siguiente comando:

```
sqoop job --exec myjob
```

El cual iniciaría la importación de la tabla `alumnos` en la base de datos indicada. Además de la opción `--create`, existen otras opciones disponibles para el comando `job`, como son `--delete` que permite eliminar un job por medio de su identificador o su nombre, `--exec` que, como se ha visto anteriormente, permite ejecutar un job determinado, `--show` que muestra la información de configuración de un job dado y `--list` que muestra por pantalla un listado con todos los jobs disponibles los cuales, como se ha indicado anteriormente, están disponibles para todos los usuarios del cluster.

Así pues, el siguiente comando muestra los jobs disponibles así como el resultado que se ofrece por la consola.

```
sqoop job --list
Available jobs:
myjob
```

Mientras que el siguiente comando muestra cómo imprimir por pantalla la configuración de un determinado job, así como el resultado de la ejecución del comando.

```
sqoop job --show myjob
Job: myjob
Tool: import
Options:
-----
direct.import = false
codegen.input.delimiters.record = 0
hdfs.append.dir = false
db.table = mytable
...
```